

IOT · SMART FEEDER

Manual PetFeeder

Dispensador automático de croquetas y agua para mascotas
App Android + API (.NET) + Prototipo ESP32

Versión del documento: 01/07/2026

Contenido

1. Qué es PetFeeder y cómo está armado
2. Requisitos previos
3. Poner en marcha el servidor (API + Base de Datos)
4. Instalar la app en un teléfono real (USB) — IP automática
5. Conexión a la red / servidor (automático y manual)
6. Autenticación: registro, verificación (OTP) y login
7. Mascotas y porciones por raza, edad y tamaño
8. Horarios de comida
9. Dispensar comida manualmente
10. Módulo de Agua (horarios y dispensado manual)
11. Conexión con el prototipo ESP32
12. Historial
13. Perfil y cerrar sesión
14. Modo sin conexión (offline)
15. Solución de problemas
16. Notas técnicas y pendientes

1. Qué es PetFeeder y cómo está armado

PetFeeder es un sistema para alimentar y dar agua a tu mascota de forma automática y remota. Se compone de tres piezas:

Pieza	Tecnología	Función
App móvil	Android (Kotlin)	Controlas mascotas, horarios, dispensado y ves el historial.
API / servidor	ASP.NET Core 8 + SQL Server	Guarda todo en la base <code>petfeeder_db</code> y manda el correo de verificación.
Prototipo	ESP32 + servo + báscula (HX711)	Suelta la croqueta/agua físicamente cuando la app o un horario lo ordena.

Flujo de datos: App → API → SQL Server (`petfeeder_db`). Para dispensar físico: App → ESP32 (por WiFi, HTTP directo).

2. Requisitos previos

- PC con **Visual Studio 2022** y **.NET 8 SDK** (para la API).
- **SQL Server** con la base `petfeeder_db` ya creada.
- **Android Studio** (para compilar e instalar la app).
- Un teléfono Android con **Depuración USB** activada.
- Una red **WiFi** a la que se conecten la PC y el teléfono (y el ESP32).
- Cuenta de Gmail con verificación en 2 pasos y una **contraseña de aplicación** (ya configurada para enviar los OTP).

3. Poner en marcha el servidor (API + Base de Datos)

1. Abre **SQL Server** y confirma que existe la base `petfeeder_db` con sus tablas (usuarios, mascotas, horarios, horarios_agua, dispensaciones, dispensaciones_agua, etc.).
2. Abre `PetFeeder.API.sln` en Visual Studio.
3. Verifica que exista `appsettings.json` con tu cadena de conexión y tu App Password de Gmail (sección `EmailSettings`).
4. Presiona **F5** (perfil **https**). La API queda escuchando en:
 - Swagger (pruebas): `https://localhost:7127/swagger`
 - Para el teléfono: `http://IP_DE_TU_PC:5172` (escucha en 0.0.0.0)

Firewall (una sola vez por PC). Abre PowerShell como Administrador y ejecuta:

```
New-NetFirewallRule -DisplayName "PetFeeder API 5172" -Direction Inbound -Protocol TCP -LocalPort 5172 -Action Allow
```

4. Instalar la app en un teléfono real (USB) — IP automática

La app detecta **sola** la IP de tu PC al compilar, así que no tienes que escribir ninguna dirección. Cada vez que compilas, se "hornea" la IP actual.

1. En el teléfono: Ajustes → *Acerca del teléfono* → toca 7 veces "Número de compilación" para activar **Opciones de desarrollador**.
2. En Opciones de desarrollador, activa **Depuración USB**.
3. Conecta el teléfono por USB y acepta el diálogo "¿Permitir depuración USB?".
4. En Android Studio, selecciona tu teléfono y pulsa **Run ▶**.

Cambio de red (presentación): ya en el lugar, conéctate a esa WiFi con la PC y el teléfono, y vuelve a pulsar **Run ▶**. La app tomará la nueva IP automáticamente. No hay que escribir nada.

Importante: compila (Run) estando ya en la WiFi del lugar. Si compilas en casa y solo llevas el teléfono sin recompilar allá, quedaría la IP de casa.

5. Conexión a la red / servidor

Automático (recomendado): no haces nada; la app usa la IP detectada al compilar.

Manual (opcional): en la pantalla de Login toca  **Configurar servidor**. Ahí puedes:

- Escribir una IP a mano, ej. `http://192.168.1.50:5172/` y pulsar "Guardar manual".
- Volver al modo automático con "Usar automático".

La IP de tu PC la ves en Windows con `ipconfig` (campo "Dirección IPv4").

6. Autenticación: registro, verificación (OTP) y login

Registrarse

1. En Login toca "Regístrate".
2. Llena nombre, correo, teléfono (opcional) y contraseña. Pulsa continuar.
3. La API crea tu cuenta (contraseña encriptada con BCrypt) y te envía un **código de 6 dígitos** por correo.

Verificar (OTP)

1. Revisa tu correo (y spam). Escribe el código de 6 dígitos.
2. El código expira en unos minutos; si caduca, solicita otro.
3. Al validar, tu cuenta queda verificada y puedes iniciar sesión.

Iniciar sesión

Escribe correo y contraseña. Si son correctos y la cuenta está verificada, entras al inicio.

Control de acceso: al cerrar sesión no puedes "regresar" con el botón atrás; hay que volver a iniciar sesión. Si ya tienes sesión, la app entra directo (auto-login).

7. Mascotas y porciones por raza, edad y tamaño

En la pestaña **Mascotas** puedes agregar, editar y eliminar mascotas. Al crear/editar eliges:

- **Raza** (selector): define el tamaño (pequeño, mediano, grande, gigante).
- **Edad** (selector): 1 mes ... 11 meses, y 1 año ... 20 años.
- **Peso** (kg): mejora la precisión del cálculo.
- **Foto y mascota principal** (solo una activa; es la del inicio).

¿Cómo se calcula la porción?

La app calcula localmente la ración diaria como un **porcentaje del peso corporal** que depende de la **etapa de vida** (según la edad). A menor edad, mayor porcentaje (los cachorros comen más en proporción):

Edad	% del peso/día	Comidas/día	Etapa
1–3 meses	~8%	4	Cachorro
4–5 meses	~6%	3	Cachorro
6–8 meses	~4.5%	3	Cachorro
9–11 meses	~3.5%	2	Cachorro
1–6 años	~2.5%	2	Adulto
7 años o más	~2.2%	2	Senior

Ejemplo: cachorro de 3 meses y 9 kg → ~720 g/día en 4 comidas de ~180 g. Si no capturas peso, se usa un peso promedio según el tamaño de la raza.

Recomendación de proteína

La tarjeta de recomendación sugiere además complementar con proteína (p. ej. pollo cocido sin hueso para cachorros; proteína magra para senior). Esta porción calculada se usa como valor por defecto al **dispensar** y al **crear horarios**.

8. Horarios de comida

1. Ve a la pestaña **Horarios** y pulsa "Nuevo horario".
2. Elige nombre (Desayuno, Cena...), hora, días de la semana y porción (viene prellenada con la recomendada).
3. Guarda. El horario aparece en la lista con un switch para activarlo/desactivarlo.

Dispensado automático: a la hora programada de cada día activo, la app le ordena al ESP32 dispensar esa porción y guarda el evento en el historial. El horario se repite cada semana.

La tarjeta "Total diario" muestra la suma de gramos activos vs. lo recomendado para la mascota principal.

9. Dispensar comida manualmente

1. Desde el inicio, entra a "Dispensar ahora".
2. Ajusta la cantidad con – / + o los presets. Viene con la porción recomendada de tu mascota.
3. **Mantén presionado** el botón 2 segundos para dispensar.

Si el ESP32 está configurado, se manda la orden real; si no, se registra en modo simulado. En ambos casos se guarda en el historial.

10. Módulo de Agua

Desde **Horarios**, el **botón flotante azul con gota** (abajo a la derecha) abre **Horarios de Agua**, que funciona igual que los de comida pero en **mililitros (ml)**.

- Crea horarios de agua (Mañana, Tarde...), con hora, días y cantidad en ml.
- El recomendado de agua se estima en ~55 ml por kg de peso al día.
- El botón flotante de esa pantalla abre **Dispensar agua manual** (presets 100/200/300 ml, mantener presionado para dispensar).

Todo se guarda en las tablas nuevas `horarios_agua` y `dispensaciones_agua`.

11. Conexión con el prototipo ESP32

El ESP32 levanta un servidor HTTP en el puerto 80 con las rutas `/dispense`, `/weight`, `/status`, `/tare`. La app le habla **directo por WiFi**.

Pasos

1. En el firmware del ESP32, pon tu red WiFi:

```
const char* ssid = "TU_SSID";  
const char* password = "TU_PASSWORD";
```

(la MISMA WiFi a la que conectas el teléfono).

2. Sube el código y abre el **Monitor Serie** (115200 baudios). Al conectar imprime la IP, por ejemplo:

```
WiFi conectado  
IP: 192.168.1.42
```

Anota esa IP.

3. En la app: pestaña **Equipo** → "Vincular dispositivo" (nombre y código cualquiera) → "Conectar por WiFi" → escribe la **IP del ESP32**.
4. Si el ESP32 responde, verás "¡Conectado al dispositivo!" y queda "En línea".

Después de esto, "Dispensar ahora" y los horarios envían la orden real al ESP32, que abre el servo hasta llegar al peso objetivo.

Agua en hardware: el prototipo actual tiene un solo actuador (servo + báscula) para croqueta. El agua usa el mismo endpoint `/dispense`. Para agua real se necesita una bomba/válvula y un endpoint aparte en el ESP32.

12. Historial

En **Perfil** → **Historial** ves los dispensados agrupados por día (Hoy, Ayer, fechas). Cada registro muestra hora, si fue manual o automático y la cantidad. El historial se lee de `petfeeder_db` y, sin conexión, de la caché local.

13. Perfil y cerrar sesión

En **Perfil** ves tu nombre y correo, accedes al Historial y puedes **Cerrar sesión**. Al cerrar sesión se limpia tu sesión y regresas al Login (no se puede volver atrás sin iniciar sesión otra vez).

14. Modo sin conexión (offline)

La app es **offline-first**: si la API está disponible, todo se guarda en `petfeeder_db` (fuente de verdad) y se refresca la caché local. Si no hay conexión, la app sigue funcionando con la **caché local (SQLite)** del teléfono, y sincroniza cuando la API vuelve.

15. Solución de problemas

Síntoma	Causa probable	Solución
"No se pudo conectar con el servidor"	API apagada o distinta WiFi	Corre la API (F5); verifica que PC y teléfono estén en la misma red; recompila para tomar la IP.
No llega el correo OTP	App Password de Gmail o spam	Revisa spam; confirma <code>EmailSettings</code> en <code>appsettings.json</code> .
El teléfono no ve la API	Firewall cerrado	Ejecuta la regla de firewall (sección 3).
ESP32 "no responde"	IP incorrecta o distinta WiFi	Revisa la IP en el Monitor Serie; misma WiFi.
La app quedó con IP vieja	Compilaste en otra red	Recompila (Run) en la red actual, o usa  Configurar servidor.

16. Notas técnicas y pendientes

Base de datos: `petfeeder_db` (SQL Server). Tablas nuevas de esta versión: `horarios_agua` , `dispensaciones_agua` ; columna nueva `mascotas.edad_meses` . Script: `PetFeeder.API/database/migracion_agua_y_edad_meses.sql` .

Endpoints API principales: `/api/auth/*` , `/api/mascotas` , `/api/horarios` , `/api/horariosagua` , `/api/dispensaciones` , `/api/dispensacionesagua` .

Pendiente de seguridad (importante para producción): los endpoints de datos aún no exigen token JWT. Antes de un uso real hay que emitir JWT en el login y proteger los endpoints con autorización. También queda pendiente la sincronización offline robusta (cola de pendientes) y la válvula de agua real en el hardware.